

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Working with NoSQL MongoDB

Table of Contents

Learning Outcome:	3
1. Introduction to MongoDB	4
NoSQL Databases	4
Features and Benefits of MongoDB	4
MongoDB Data Model	5
2. Connecting to MongoDB Database	6
MongoDB Installation and Setup	6
Configuring MongoDB Connection Properties in Spring Applications	6
Establishing a Connection to MongoDB using MongoClient:	7
Handling Authentication and Authorization	8
3. Working with MongoTemplate	9
MongoTemplate and its role in Spring applications	9
Performing CRUD Operations using MongoTemplate	10
Querying and Updating Documents in MongoDB	11
Executing Aggregation Pipelines with MongoTemplate	12
4. Spring Data MongoDB Repositories	13
Spring Data MongoDB	13
Defining Domain Models and Repositories	14
Basic CRUD Operations with Spring Data MongoDB Repositories	14
Querying MongoDB using Custom Repository Methods	15
5. Testing with Embedded Mongo	16
Configuring and Setting up Embedded Mongo	16
Writing Integration Tests with Embedded Mongo	17
Test-Driven Development (TDD) with Embedded Mongo	18

UNIT 6: Working with NoSQL MongoDB

Objectives

In this Unit you will learn –

- The fundamentals of MongoDB and its benefits as a NoSQL database.
- To establish a connection to a MongoDB database from a Spring application.
- To utilize the MongoTemplate to perform CRUD (Create, Read, Update, Delete) operations in MongoDB.
- Leveraging Spring Data MongoDB repositories for simplified database access.
- To configure and use embedded Mongo for testing MongoDB interactions.

Learning Outcome:

Learn about NoSQL MongoDB

6. Working with NoSQL MongoDB

1. Introduction to MongoDB

MongoDB is a popular open-source NoSQL (Not Only SQL) database that provides a flexible and scalable approach to storing and managing data. Unlike traditional relational databases, MongoDB follows a document-oriented model, where data is stored in flexible, JSON-like documents instead of structured tables. This flexibility allows MongoDB to handle diverse and evolving data structures.

NoSQL Databases

NoSQL databases are designed to handle large volumes of unstructured or semi-structured data. They provide an alternative to traditional relational databases, offering greater scalability, flexibility, and performance for certain use cases. NoSQL databases are commonly used in modern web and mobile applications, big data processing, real-time analytics, and other scenarios where traditional databases may struggle to meet the demands.

Features and Benefits of MongoDB

1. Document-Oriented: MongoDB stores data in flexible, self-describing documents called BSON (Binary JSON), which can contain nested structures and arrays. This makes it easy to represent complex data and adapt to evolving schemas.
2. Scalability and Performance: MongoDB is designed to scale horizontally across multiple servers, allowing for high availability and accommodating growing data and traffic loads. It also supports sharding, distributing data across multiple machines for improved performance.

3. Flexible Data Model: MongoDB's document model allows for dynamic and polymorphic data structures. It does not require predefined schemas, enabling developers to easily modify and extend their data models as needed.

4. Rich Query Language: MongoDB provides a powerful query language that supports rich queries, including filtering, sorting, aggregation, and geospatial queries. It allows developers to express complex queries in a straightforward manner.

5. Indexing and High-Performance Reads: MongoDB supports various types of indexes, including single-field, compound, geospatial, and text indexes, enabling efficient querying and high-performance reads.

6. High Availability and Fault Tolerance: MongoDB offers built-in replication, ensuring data redundancy and high availability. It can automatically handle failover and maintain data consistency in the event of node failures.

MongoDB Data Model

MongoDB organizes data into collections, which are analogous to tables in relational databases. Each collection consists of multiple documents, which are JSON-like structures representing individual records. Documents within a collection do not need to have the same structure, allowing for schema flexibility.

Key concepts in the MongoDB data model include:

- Collections: Equivalent to tables in relational databases, collections are containers for documents.
- Documents: JSON-like structures that store data and represent individual records. Documents are analogous to rows in relational databases.

- Fields: Key-value pairs within a document that store specific data elements.
- Embedded Documents: Documents can contain other documents as fields, allowing for nesting and representing relationships between data.
- Arrays: Fields can contain arrays, enabling the storage of multiple values for a given field.
- BSON: Binary representation of JSON-like documents used by MongoDB.

The document-oriented data model in MongoDB provides flexibility in handling complex data structures and facilitates efficient querying and manipulation of data.

2. Connecting to MongoDB Database

MongoDB Installation and Setup

- Download the MongoDB Community Server from the official MongoDB website (<https://www.mongodb.com/>).
- Install MongoDB according to the instructions provided for your operating system.
- Configure the MongoDB server by specifying the data directory, log directory, and other settings as per your requirements.
- Start the MongoDB server to make it available for connections.

Configuring MongoDB Connection Properties in Spring Applications

- Add the MongoDB Java driver dependency to your Spring project's build configuration (e.g., Maven or Gradle).
- In your Spring application's configuration file (e.g., `application.properties` or `application.yml`), specify the MongoDB connection properties:
- `spring.data.mongodb.uri`: The MongoDB connection URI, including the hostname, port, database name, and any additional options.

Proprietary content. All rights reserved. Unauthorized use or distribution prohibited.

- `spring.data.mongodb.host`: The MongoDB server hostname.
- `spring.data.mongodb.port`: The MongoDB server port.
- `spring.data.mongodb.database`: The name of the MongoDB database to connect to.
- Additional properties like username and password for authentication (if required) can be specified.

Establishing a Connection to MongoDB using MongoClient:

- In your Spring application, create a MongoDB client bean using the `MongoClient` class provided by the MongoDB Java driver.
- The MongoClient can be configured with the MongoDB connection properties set in the previous step.
- The MongoClient is responsible for establishing and managing connections to the MongoDB server.

Example configuration in Spring Boot:

```
import com.mongodb.client.MongoClients;
```

```
import com.mongodb.client.MongoClient;
```

```
@Configuration
```

```
public class MongoDBConfig {
```

```
    @Value("${spring.data.mongodb.uri}")
```

```
    private String mongoUri;
```

```
    @Bean
```

```
    public MongoClient mongoClient() {
```

```
return MongoClient.create(mongoUri);
```

```
}
```

```
}
```

Handling Authentication and Authorization

- If your MongoDB server requires authentication, provide the username and password in the MongoDB connection URI or configure them separately in the Spring application's properties.
- In the MongoDB server, create user accounts with appropriate roles and permissions for accessing the desired databases.
- When connecting to the MongoDB server, the MongoClient will authenticate using the provided credentials if authentication is enabled.
- Ensure that the authenticated user has the necessary privileges to perform the required operations on the target database and collections.

Example MongoDB connection URI with authentication:

```
spring.data.mongodb.uri=mongodb://username:password@localhost:27017/mydatabase
```

Example Spring Boot application.properties for authentication:

```
spring.data.mongodb.host=localhost
```

```
spring.data.mongodb.port=27017
```

```
spring.data.mongodb.database=mydatabase
```

```
spring.data.mongodb.username=username
```

```
spring.data.mongodb.password=password
```


By configuring the MongoDB connection properties and establishing a connection using the MongoClient, your Spring application can interact with the MongoDB database. Ensure that the authentication and authorization settings are correctly configured to secure access to the MongoDB resources.

3. Working with MongoTemplate

MongoTemplate and its role in Spring applications

MongoTemplate is a class provided by Spring Data MongoDB that serves as a high-level abstraction for interacting with MongoDB databases in Spring applications. It simplifies the usage of MongoDB by providing convenient methods to perform CRUD operations, execute queries, and manage transactions. MongoTemplate is built on top of the MongoDB Java driver and provides a Spring-friendly approach to working with MongoDB.

Key features and roles of MongoTemplate in Spring applications include:

1. **Encapsulating MongoDB Operations:** MongoTemplate encapsulates the low-level details of interacting with MongoDB, allowing developers to focus on business logic instead of dealing with raw MongoDB Java driver code.
2. **Mapping Documents to Java Objects:** MongoTemplate facilitates mapping MongoDB documents to Java objects and vice versa. It provides mechanisms for converting between BSON (Binary JSON) documents and Java objects, enabling seamless integration between the document-oriented nature of MongoDB and object-oriented programming in Spring applications.
3. **Simplified CRUD Operations:** MongoTemplate provides intuitive methods for performing Create, Read, Update, and Delete (CRUD) operations on MongoDB collections. Developers can easily save, update, retrieve, and delete documents using familiar Spring patterns.

4. Querying and Updating Documents: MongoTemplate supports querying and updating documents using a variety of criteria, such as exact matches, regular expressions, ranges, and more. It allows developers to specify complex conditions to retrieve or modify data from MongoDB collections.

5. Aggregation Pipelines: MongoTemplate offers support for MongoDB's powerful aggregation framework, which allows for advanced data processing and analytics. It enables developers to construct and execute aggregation pipelines, combining multiple stages to perform complex data transformations, grouping, filtering, and analysis.

Performing CRUD Operations using MongoTemplate

MongoTemplate provides methods for performing CRUD operations on MongoDB collections. Some commonly used methods include:

1. Saving Documents:

- ``mongoTemplate.save(object, collectionName)``: Saves a document to the specified collection.
- ``mongoTemplate.insert(object, collectionName)``: Inserts a document into the specified collection.

2. Retrieving Documents:

- ``mongoTemplate.findById(id, entityClass, collectionName)``: Retrieves a document by its ID from the specified collection.
- ``mongoTemplate.find(query, entityClass, collectionName)``: Retrieves documents based on the provided query criteria.
- ``mongoTemplate.findAll(entityClass, collectionName)``: Retrieves all documents from the specified collection.

3. Updating Documents:

- ``mongoTemplate.updateFirst(query, update, entityClass, collectionName)``: Updates the first document matching the query criteria.
- ``mongoTemplate.updateMulti(query, update, entityClass, collectionName)``: Updates multiple documents matching the query criteria.
- ``mongoTemplate.findAndModify(query, update, entityClass, collectionName)``: Finds a document matching the query criteria, updates it, and returns the modified document.

4. Deleting Documents:

- ``mongoTemplate.remove(query, entityClass, collectionName)``: Removes documents matching the query criteria.
- ``mongoTemplate.remove(object, collectionName)``: Removes a specific document from the specified collection.

Querying and Updating Documents in MongoDB

MongoTemplate provides a Query class that allows developers to define query criteria and options for retrieving or modifying documents. Some common querying and updating methods include:

1. Querying:

- ``Query query = new Query()``: Creates a new query object.
- ``query.addCriteria(Criteria.where("field").is(value))``: Adds a criteria to match documents where the specified field equals a specific value.
- ``query.limit(n)``: Limits the number of documents to retrieve.
- ``query.with(Sort.by(Sort.Direction.ASC, "field"))``: Specifies the sorting order for retrieved documents.

2. Updating:

- `Update update = new Update()`: Creates a new update object.
- `update.set("field", value)`: Sets the value of a field in the document.
- `update.inc("field", value)`: Increments the value of a numeric field.
- `update.addToSet("field", value)`: Adds an element to an array field if it doesn't already exist.

Executing Aggregation Pipelines with MongoTemplate

MongoTemplate provides methods for executing aggregation pipelines, which consist of multiple stages to process and analyze data. Some commonly used aggregation methods include:

1. `mongoTemplate.aggregate(Aggregation aggregation, String inputCollection, Class<?> outputType)`: Executes an aggregation pipeline and returns the result mapped to the specified outputType.

- Aggregation stages can be defined using the Aggregation class, including stages like `$match`, `$group`, `$project`, `$sort`, and more.

2. `AggregationOperation` and `TypedAggregation`: These classes can be used to define complex aggregation pipelines by chaining multiple stages together.

The MongoTemplate, along with its various methods and query-building capabilities, provides a powerful and convenient way to interact with MongoDB in Spring applications, enabling developers to work with MongoDB's document model while leveraging Spring's features and patterns.

4. Spring Data MongoDB Repositories

Spring Data MongoDB

Spring Data MongoDB is a module of the Spring Data project that provides a high-level abstraction for working with MongoDB in Spring applications. It simplifies the development of MongoDB-based applications by eliminating boilerplate code and providing convenient APIs for data access and manipulation. Spring Data MongoDB leverages the MongoDB Java driver and integrates seamlessly with other Spring technologies.

Key features of Spring Data MongoDB include:

1. **Automatic Repository Implementation:** Spring Data MongoDB automatically generates the implementation of repositories based on defined interfaces. This eliminates the need to write repetitive CRUD operations and simplifies database interactions.
2. **Query Methods:** Spring Data MongoDB allows you to define query methods in repository interfaces using a naming convention. These methods can automatically generate MongoDB queries based on their names, simplifying the process of querying and filtering data.
3. **MongoDB Template Integration:** Spring Data MongoDB integrates with the MongoTemplate, providing a lower-level, more flexible API for interacting with MongoDB. It allows you to combine the convenience of repository methods with the flexibility of custom queries using the MongoTemplate.
4. **Support for Aggregations:** Spring Data MongoDB provides support for executing MongoDB aggregation pipelines through the Aggregation API. It enables developers to perform advanced data processing, grouping, and analysis using the familiar Spring Data programming model.

Defining Domain Models and Repositories

To work with Spring Data MongoDB, you need to define domain models and repositories.

1. Domain Models:

- Define Java classes representing the data stored in MongoDB collections.
- Annotate the domain models with appropriate annotations like `@Document`, `@Field`, `@Indexed`, etc., to map them to MongoDB documents and define indexes.
- Establish relationships between domain models using annotations like `@DBRef` for references.

2. Repositories:

- Create a repository interface by extending the `MongoRepository` or `MongoCrudRepository` interface provided by Spring Data MongoDB.
- Specify the domain model type and the ID type (usually `String` or `ObjectId`) as generic parameters.
- Spring Data MongoDB will automatically generate the implementation for basic CRUD operations based on the repository interface.

Basic CRUD Operations with Spring Data MongoDB Repositories

Spring Data MongoDB repositories provide default implementations for basic CRUD operations, eliminating the need to write boilerplate code. Some common methods available in repositories include:

1. `save(entity)`: Saves a document (entity) to the collection. It performs an insert or update operation based on the existence of the document.

2. `findById(id)`: Retrieves a document by its ID.
3. `findAll()`: Retrieves all documents from the collection.
4. `delete(entity)`: Deletes a document from the collection.
5. `deleteById(id)`: Deletes a document by its ID.
6. `existsById(id)`: Checks if a document with the given ID exists in the collection.

Querying MongoDB using Custom Repository Methods

Spring Data MongoDB repositories support custom query methods, allowing you to define specific queries without writing explicit query code. Some approaches to define custom queries include:

1. Query Methods: Spring Data MongoDB derives queries from method names by following a naming convention. For example:
 - `findByField(value)`: Retrieves documents by a specific field value.
 - `findByFieldAndAnotherField(value1, value2)`: Retrieves documents by multiple field values.
 - `findByFieldIn(values)`: Retrieves documents where the field matches any value from a given list.
2. Query Annotations: Spring Data MongoDB provides annotations like `@Query` and `@QueryAnnotation` that allow you to define custom queries using MongoDB query language or JSON-based queries.

3. QueryDSL: Spring Data MongoDB integrates with QueryDSL, a type-safe query framework. It enables you to define complex queries using a fluent API.

By leveraging custom repository methods, you can easily define and execute queries on MongoDB collections without writing complex query code manually.

Spring Data MongoDB, with its repository model, provides a powerful and convenient way to interact with MongoDB in Spring applications. It promotes code reuse, simplifies database interactions, and integrates seamlessly with other Spring technologies, enabling efficient and streamlined MongoDB data access.

5. Testing with Embedded Mongo

Embedded Mongo is a lightweight, in-memory version of the MongoDB database that is specifically designed for testing and development purposes. It allows you to run MongoDB in an isolated environment without the need for a separate MongoDB server installation. Embedded Mongo is especially useful for writing integration tests, as it provides a fast and controlled environment for testing database interactions.

Configuring and Setting up Embedded Mongo

To use Embedded Mongo in your tests, you need to configure and set it up appropriately. Here are the general steps involved:

1. Add the Embedded Mongo Dependency: Include the Embedded Mongo dependency in your project's build configuration (e.g., Maven or Gradle). The exact dependency will depend on the testing framework you are using, such as `de.flapdoodle.embed:de.flapdoodle.embed.mongo` for JUnit.

2. Configure Embedded Mongo Properties: Configure the properties for the Embedded Mongo instance. This typically includes specifying the port, database

name, and other configuration options. You can either use default values or customize them as per your requirements.

3. Start and Stop Embedded Mongo: In your test setup and teardown methods (e.g., `@Before` and `@After` methods), start the Embedded Mongo instance before running tests and stop it after the tests have completed. This ensures that the Embedded Mongo server is up and running during the test execution.

Writing Integration Tests with Embedded Mongo

Once you have set up Embedded Mongo, you can write integration tests that interact with the embedded database. Here are the general steps involved:

1. Define Test Cases: Identify the specific test cases you want to write, focusing on the database interactions you want to test. This may include saving data, querying data, updating data, and verifying the results.

2. Prepare Test Data: In the setup phase of each test case, insert the necessary data into the embedded database. This ensures that the database is in the desired initial state for the test.

3. Perform Database Operations: In the test methods, perform the required database operations, such as saving data, retrieving data, or updating data, using the appropriate repository or data access objects.

4. Assert the Results: After performing the database operations, assert the results to verify that the expected data is returned or updated correctly. Use assertions provided by your testing framework to compare the actual results with the expected results.

5. Cleanup Data: In the teardown phase of each test case (e.g., `@After` method), clean up any data that was inserted or modified during the test. This ensures that subsequent tests start with a clean slate.

Test-Driven Development (TDD) with Embedded Mongo

Test-Driven Development (TDD) is an approach where tests are written before implementing the actual functionality. When using Embedded Mongo for TDD, you can follow these steps:

1. Write Test Cases: Start by writing test cases that define the expected behavior of the functionality you want to implement. These tests should be written without any implementation code.
2. Run Tests: Run the tests and verify that they fail, as the functionality is not yet implemented.
3. Implement Functionality: Write the necessary code to implement the desired functionality, ensuring that the tests pass one by one.
4. Refactor and Repeat: Refactor your code as needed to improve its design and maintainability. Once the functionality is implemented and the tests pass, repeat the process for the next set of requirements or features.

By using Embedded Mongo in your testing environment and following TDD principles, you can write reliable and robust integration tests that validate the behavior of your MongoDB interactions, ensuring the correctness and stability of your application.